Dart Programming

Operators

- Operators are used for doing operations on values and variables
- Dart have several categories of operators
  - Arithmetic operators
  - Equality and relational operators
  - Type test operators
  - Assignment operators
  - Logical operators

Arithmetic Operators

- Dart supports the usual arithmetic operators, as shown in the following table.

Operator	Meaning
`+`	Add
`-`	Subtract
`-` *expr*	Unary minus, also known as negation (reverse the sign of the expression)
`*`	Multiply
`/`	Divide
`~/`	Divide, returning an integer result
`%`	Get the remainder of an integer division (modulo)

Arithmetic Operators

- Examples
- 2 + 3 // output = 5
- 2 - 3 // output = -1
- 2 *3 // output = 6
- 5 / 2 // Result is a double 2.5
- 5 ~/ 2; // Result is an int 2
- 5 % 2 // Remainder is 1
- Dart also supports both prefix and postfix increment and decrement operators.

Operator	Meaning
`++var`	`var = var + 1` (expression value is `var + 1`)
`var++`	`var = var + 1` (expression value is `var`)
`--var`	`var = var - 1` (expression value is `var - 1`)
`var--`	`var = var - 1` (expression value is `var`)

Arithmetic Operators


```dart
// Dart Program Demonstrating use
// Of all Arithmetic Operators

void main()
{
    int a = 2;
    int b = 3;

    // Adding a and b
    var c = a + b;
    print("Sum  (a + b) = $c");

    // Subtracting a and b
    var d = a - b;
    print("Difference (a - b) = $d");

    // Using unary minus
    var e = -d;
    print("Negation -(a - b) = $e");

    // Multiplication of a and b
    var f = a * b;
    print("Product (a * b) = $f");

    // Division of a and b
    var g = b / a;
    print("Division (b / a) = $g");

    // Using ~/ to divide a and b
    var h = b ~/ a;
    print("Quotient (b ~/ a) = $h");

    // Remainder of a and b
    var i = b % a;
    print("Remainder (b % a) = $i");
}
```

Equality & Relational Operators

- The following table lists the meanings of equality and relational operators.

Operator Symbol	Operator Name	Operator Description
>	Greater than	Check which operand is bigger and give result as boolean expression.
<	Less than	Check which operand is smaller and give result as boolean expression.
>=	Greater than or equal to	Check which operand is greater or equal to each other and give result as boolean expression.
<=	less than equal to	Check which operand is less than or equal to each other and give result as boolean expression.
==	Equal to	Check whether the operand are equal to each other or not and give result as boolean expression.
!=	Not Equal to	Check whether the operand are not equal to each other or not and give result as boolean expression.

Example


```dart
// Dart Program Demonstrating use
// Of all Relational Operators
void main()
{
    int a = 2;
    int b = 3;

    // Greater between a and b
    var c = a > b;
    print("a is greater than b (a > b) : $c");

    // Smaller between a and b
    var d = a < b;
    print("a is smaller than b (a < b) : $d");

    // Greater than or equal to between a and b
    var e = a >= b;
    print("a is greater than b (a >= b) : $e");

    // Less than or equal to between a and b
    var f = a <= b;
    print("a is smaller than b (a <= b) : $f");

    // Equality between a and b
    var g = b == a;
    print("a and b are equal (b == a) : $g");

    // Unequality between a and b
    var h = b != a;
    print("a and b are not equal (b != a) : $h");
}
```

Type test operators

Type test operators

The `as`, `is`, and `is!` operators are handy for checking types at runtime.

Operator	Meaning
`as`	Typecast (also used to specify library prefixes)
`is`	True if the object has the specified type
`is!`	True if the object doesn't have the specified type

Int a = 5 ;

a is double // false
a is int // true

Example


```
void main()
{
    String a = 'GFG';
    double b = 3.3;

    // Using is to compare
    print(a is String);

    // Using is! to compare
    print(b is !int);
}
```

```
// Dart Program to demonstrate
// Use of as Operator

void main(){
  // Declaring value
  dynamic value = "Hello";

  // TypeCast dynamic -> String
  String str= value as String;

  // Print String
  print(str);
}
```

Assignment Operator

- As you've already seen, you can assign values using the = operator. To assign only if the assigned-to variable is null, use the ??= operator.

```
// Assign value to a
a = value;
// Assign value to b if b is null; otherwise, b stays the same
b ??= value;
```

- Compound assignment operators such as += combine an operation with an assignment.

```
var a = 2; // Assign using =
a *= 3; // Assign and multiply: a = a * 3
assert(a == 6);
```

Example


```
void main()
{
    int a = 5;
    int b = 7;

    // Assigning value to variable c
    var c = a * b;

    print("assignment  operator used c = a*b so now c = $c\n");

    // Assigning value to variable d
    var d;

      // Value is assign as it is null
    d ??= a + b;

    print("Assigning value only if d is null");
    print("d??= a+b so d = $d \n");

    // Again trying to assign value to d
    d ??= a - b;
      // Value is not assign as it is not null

    print("Assigning value only if d is null");
    print("d??= a-b so d = $d");
    print("As d was not null value was not updated");
}
```

Logical Operators

- This class of operators contain those operators which are used to logically combine two or more conditions of the operands. It goes like this

Operator Symbol	Operator Name	Operator Description
&&	And Operator	Use to add two conditions and if both are true than it will return true.
||	Or Operator	Use to add two conditions and if even one of them is true than it will return true.
!	Not Operator	It is use to reverse the result.

Example


```
void main()
{
    int a = 5;
    int b = 7;

    // Using And Operator
    bool c = a > 10 && b < 10;
    print(c);

    // Using Or Operator
    bool d = a > 10 || b < 10;
    print(d);

    // Using Not Operator
    bool e = !(a > 10);
    print(e);
}
```

Example


```dart
// Dart Program to demonstrate use of
// Logical Operators

void main()
{
    var a = true;
    var b = false;
      // Printing the Values of a & b
    print("a: $a , b: $b\n");

    // Using And Operator
    print("a && b = ${a&&b}");

    // Using Or Operator
    print("a || b = ${a||b}");

    // Using Not Operator
    print("!a = ${!a}");
}
```

Conditional Operators

- This class of operators contain those operators which are used to perform comparison on the operands. It goes like this:

Operator Symbol	Operator Name	Operator Description
condition ? expression1 : expersion2	Conditional Operator	It is a simple version of if-else statement. If the condition is true than expersion1 is executed else expersion2 is executed.
expersion1 ?? exppression2	Conditional Operator	If expersion1 is non-null returns its value else returns expersion2 value.

Example

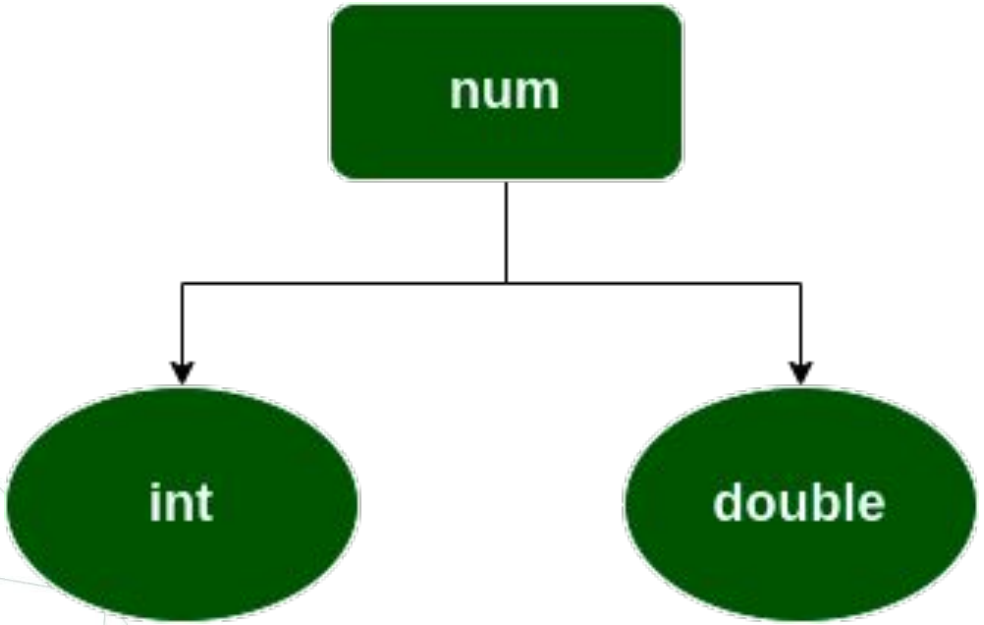

```
void main()
{
    int a = 5;

    // Conditional Statement
    var c = (a < 10) ? "Statement is Correct, Geek" : "Statement is Wrong, Geek";
    print(c);


    // Conditional statement
      int? n;
      // Warning: Operand of null-aware operation '??' has type 'int' which excludes null.
      // For batter practice make both same type to avoid warning
      // var d = n ?? 10;
      var d = n ?? "n has Null value";
      print(d);

    // After assigning value to n
    n = 10;
      // we make it all ready null safe
    //d = n ? ? "n has Null value";
      d = n;
    print(d);
}
```

Numbers

- The number in Dart Programming is the data type that is used to hold the numeric value. Dart numbers can be classified as:
- int: data type is used to represent whole numbers (64-bit Max).
- double: data type is used to represent 64-bit precise floating-point numbers.
- num: type is an inherited data type of the int and double types.

```dart
// Dart Program to demonstrate
// Number Data Type

void main() {

   // declare an integer
   int num1 = 2;

   // declare a double value
   double num2 = 1.5;

   // print the values
   print(num1);
   print(num2);
   var a1 = num.parse("1");
   var b1 = num.parse("2.34");

   var c1 = a1+b1;
   print("Product = ${c1}");
}
```

Basics of Numbers



- int: The int data type is used to represent whole numbers.
- double: The double data type is used to represent 64-bit floating-point numbers.
- Note: The num type is an inherited data type of the int and double types.
- Parsing in Dart: The parse() function is used parsing a string containing numeric literal and convert to the number.

```
void main()
{

  var a1 = num.parse("1");
  var b1 = num.parse("2.34");

  var c1 = a1+b1;
  print("Product = ${c1}");
}
```

Numbers Properties

- hashcode: This property is used to get the hash code of the given number.
- isFinite: If the given number is finite, then this property will return true.
- isInfinite: If the number is infinite, then this property will return true.
- isNan: If the number is non-negative then this property will return true.
- isNegative: If the number is negative then this property will return true.
- sign: This property is used to get -1, 0, or 1 depending upon the sign of the given number.
- isEven: If the given number is an even then this property will return true.
- isOdd: If the given number is odd then this property will return true.

Numbers Methods

- abs(): This method gives the absolute value of the given number.
- ceil(): This method gives the ceiling value of the given number.
- floor(): This method gives the floor value of the given number.
- compareTo(): This method compares the value with other numbers.
- remainder(): This method gives the truncated remainder after dividing the two numbers.
- round(): This method returns the round of the number.
- toDouble(): This method gives the double equivalent representation of the number.
- toInt(): This method returns the integer equivalent representation of the number.
- toString(): This method returns the String equivalent representation of the number
- truncate(): This method returns the integer after discarding fraction digits.

Strings

رواد مصر الرقمية

وزارة الاتصـــــالات
وتكنولوجيا المعلومات

- A Dart string (String object) holds a sequence of UTF-16 code units. You can use either single or double quotes to create a string:

```dart
var s1 = 'Single quotes work well for string literals.';
var s2 = "Double quotes work just as well.";
var s3 = 'It\'s easy to escape the string delimiter.';
var s4 = "It's even easier to use the other delimiter.";
```

- You can put the value of an expression inside a string by using ${expression}. If the expression is an identifier, you can skip the {}. To get the string corresponding to an object, Dart calls the object's toString() method.
- This approach called String Interpolation.

String Interpolation Example

```dart
var s = 'string interpolation';

assert('Dart has $s, which is very handy.' ==
    'Dart has string interpolation, '
        'which is very handy.');
assert('That deserves all caps. '
        '${s.toUpperCase()} is very handy!' ==
    'That deserves all caps. '
        'STRING INTERPOLATION is very handy!');
```

String Concatenation

- You can concatenate strings using adjacent string literals or the + operator:

```dart
var s1 = 'String '
    'concatenation'
    " works even over line breaks.";
assert(s1 ==
    'String concatenation works even over '
        'line breaks.');

var s2 = 'The + operator ' + 'works, as well.';
assert(s2 == 'The + operator works, as well.');
```

Multi-line String

- To create a multi-line string, use a triple quote with either single or double quotation marks:

```dart
var s1 = '''
You can create
multi-line strings like this one.
''';

var s2 = """This is also a
multi-line string.""";
```

Collections

رواد مصر الرقمية

وزارة الاتصالات وتكنولوجيا المعلومات

- Dart has built-in support for List, Set, and Map collections.
- List is Perhaps the most common collection in nearly every programming language is the array, or ordered group of objects. In Dart, arrays are List objects, so most people just call them lists.
- Dart list literals are denoted by a comma separated list of expressions or values, enclosed in square brackets ([]). Here's a simple Dart list:
- var list = [1, 2, 3];
- Dart infers that list has type List<int>. If you try to add non-integer objects to this list, the analyzer or runtime raises an error.
- Lists use zero-based indexing, where 0 is the index of the first value and list.length - 1 is the index of the last value.

List

- List data type is similar to arrays in other programming languages. A list is used to represent a collection of objects. It is an ordered group of objects.
- There are multiple methods to declare List in Dart as mentioned below:
- Variable Size List

```
// Correct way to declare a variable-sized list
List<int> var_name1 = [];

// Alternative declaration
List<int> var_name2;
```

Fixed Size List

```
List<int> var_name1 = List<int>.filled(size, 0);

List<int> var_name2 = List<int>.generate(size, (index) => index * 2);
```

Sets

- A set in Dart is an unordered collection of unique items. Dart support for sets is provided by set literals and the Set type.
- var halogens = {'fluorine', 'chlorine', 'bromine', 'iodine', 'astatine’};
- Dart infers that halogens has the type Set<String>. If you try to add the wrong type of value to the set, the analyzer or runtime raises an error.
- Use .length to get the number of items in the set:
- var elements = <String>{};
- elements.add('fluorine');
- elements.addAll(halogens);
- assert(elements.length == 5);

Set

- To create a set that's a compile-time constant, add const before the set literal:

```
final constantSet = const {
  'fluorine',
  'chlorine',
  'bromine',
  'iodine',
  'astatine',
};
// constantSet.add('helium'); // This line will cause an error.
```

Maps

- In general, a map is an object that associates keys and values.
- Both keys and values can be any type of object.
- Each key occurs only once, but you can use the same value multiple times.

```
var gifts = {
  // Key:    Value
  'first': 'partridge',
  'second': 'turtledoves',
  'fifth': 'golden rings'
};

var nobleGases = {
  2: 'helium',
  10: 'neon',
  18: 'argon',
};
```

Map

- You can create the same objects using a Map constructor:

```
var gifts = Map<String, String>();
gifts['first'] = 'partridge';
gifts['second'] = 'turtledoves';
gifts['fifth'] = 'golden rings';

var nobleGases = Map<int, String>();
nobleGases[2] = 'helium';
nobleGases[10] = 'neon';
nobleGases[18] = 'argon';
```

- Add a new key-value pair to an existing map using the subscript assignment operator ([]=):
- var gifts = {'first': 'partridge'};
- gifts['fourth'] = 'calling birds'; // Add a key-value pair

Map

- Retrieve a value from a map using the subscript operator ([ ]):
- var gifts = {'first': 'partridge’}; gifts['first'] == 'partridge’ ;
- If you look for a key that isn't in a map, you get null in return:
- Use .length to get the number of key-value pairs in the map.
- var gifts = {'first': 'partridge'};
- gifts['fourth'] = 'calling birds';
- gifts.length // 2
- To create a map that's a compile-time constant, add const before the map literal:
- final constantMap = const {
- 2: 'helium',
- 10: 'neon',
- 18: 'argon',};
- // constantMap[2] = 'Helium'; // This line will cause an error.